



南開大學
Nankai University

计算机学院
并行程序设计实验报告

MPI 编程

姓名：王一诺

学号：2312331

专业：计算机科学与技术

2025 年 6 月 9 日

目录

1	实验目标	2
2	实验平台配置	2
3	MPI 常规优化	2
3.1	设计思想	2
3.2	关键代码实现	3
4	多进程多线程多级并行 (2×2)	4
4.1	设计思想	4
4.2	关键代码实现	4
5	Barrett 规约优化	7
5.1	Barrett 规约设计	7
5.2	Barrett 规约 +MPI 常规优化	9
5.3	Barrett 规约 + 线程进程多级并行	11
6	测试结果与分析	14
6.1	实验结果表格	14
6.2	结果分析	14
7	结论	14
8	链接	15

1 实验目标

本实验旨在基于快速数论变换(NTT)多项式乘法的基础实现,分别引入 MPI 多进程并行、MPI+Pthread 多级并行、Barrett 模约简优化等多种优化策略,通过对比朴素的多模数合并 NTT(NTT_CRT_4mod)实现,评估各优化策略在不同规模多项式乘法任务下的性能提升效果。具体目标包括:

1. 理解并实现基于 MPI 的常规并行 NTT (NTT_mpi), 掌握进程间数据分发与收集的基本流程。
2. 在每个 MPI 进程内部引入 Pthread 多线程, 实现 2×2 级并行 (NTT_mpi_pthread), 探索进程与线程混合并行的协同效应。
3. 应用 Barrett 模约简技术, 对 NTT 核心运算中的乘法与幂运算进行加速, 构建 NTT_mpi_barrett, 以及进一步结合 Pthread 多级并行的 NTT_mpi_pthread_barrett。
4. 对比分析各方案在输入规模为 $n = 4$ 与 $n = 131072$ 、不同模数下的运行时长与加速比, 并与基础的 NTT_CRT_4mod 结果进行对比, 揭示各优化方案的适用场景与性能瓶颈。

2 实验平台配置

平台 \ 处理器	操作系统	编译器
ARM/华为鲲鹏处理器	OpenEuler	mpic++

表 1: 实验环境

3 MPI 常规优化

3.1 设计思想

MPI 常规优化策略基于“每个进程处理一个模数”的思路,将四路模数分配给最多四个 MPI 进程并行计算。具体流程如下:

1. **模块划分:** 定义四个素数模数 p_1, p_2, p_3, p_4 , 将它们存入数组 `mods[]`。
2. **数据分发:** 每个 rank 小于 4 的进程单独加载输入多项式数据, 将其取模到对应模数空间。
3. **NTT 计算:** 在本地进程中完成 NTT 正变换、点值乘、逆变换及归一化, 得到当前模下的乘积结果。
4. **结果汇聚:** 使用 `MPI_Gather` 将各进程的局部结果收集到 rank 0 进程, 并在 rank 0 上通过 CRT 完成四路合并。
5. **广播输出:** 最终结果通过 `MPI_Bcast` 发送给所有进程, 保证一致性。

3.2 关键代码实现

Listing 1: MPI 常规优化核心函数 NTT_mpi

```

1  void NTT_mpi(const LL *a, const LL *b, LL *ab, int n, LL LLp) {
2      int rank, size;
3      MPI_Comm_rank(MPI_COMM_WORLD, &rank);
4      MPI_Comm_size(MPI_COMM_WORLD, &size);
5
6      static const int p1 = 998244353, p2 = 1004535809,
7      p3 = 469762049, p4 = 167772161;
8      const int mods[] = {p1, p2, p3, p4};
9
10     // 每个进程处理一个模数
11     if (rank < 4) {
12         int curr_mod = mods[rank];
13         // 转换输入到当前模数
14         int *local_a = new int[n];
15         int *local_b = new int[n];
16         for (int i = 0; i < n; i++) {
17             local_a[i] = (a[i] % curr_mod + curr_mod) % curr_mod;
18             local_b[i] = (b[i] % curr_mod + curr_mod) % curr_mod;
19         }
20
21         // 执行NTT乘法
22         int *local_res = new int[2*n];
23         NTT_Iteration_multiply(local_a, local_b, local_res, n, curr_mod);
24
25         // 收集结果
26         int *all_results = nullptr;
27         if (rank == 0) all_results = new int[4 * (2*n)];
28         MPI_Gather(local_res, 2*n, MPI_INT, all_results, 2*n, MPI_INT, 0,
29             MPI_COMM_WORLD);
30
31         // 主进程执行CRT合并
32         if (rank == 0) {
33             for (int i = 0; i < 2*n-1; i++) {
34                 ab[i] = crt4(all_results[i], all_results[i + 2*n],
35                     all_results[i + 4*n], all_results[i + 6*n],
36                     p1, p2, p3, p4, LLp);
37             }
38             delete[] all_results;
39
40             delete[] local_a; delete[] local_b; delete[] local_res;
41         }
42
43         // 广播最终结果
44         MPI_Bcast(ab, 2*n-1, MPI_LONG_LONG, 0, MPI_COMM_WORLD);

```

45 }

实现特点：1. 使用 MPI_Comm_rank 和 MPI_Comm_size 获取进程信息

2. 每个进程独立处理模转换和 NTT 计算

3. MPI_Gather 收集中间结果，仅在主进程执行 CRT 合并

4. MPI_Bcast 保证所有进程获得最终结果

4 多进程多线程多级并行 (2×2)

4.1 设计思想

在 MPI 常规优化的基础上，进程内再引入多线程并行，每个进程处理若干模数，每个模数分配一路线程完成 NTT 计算。总体流程：

1. 模数划分：根据进程数动态计算每个进程负责的模数列表（最多两模/进程）。

2. 线程调度：针对每个模数，创建独立线程执行 NTT 正变换、点值乘、逆变换。

3. 结果收集：采用 MPI_Gatherv 收集不定长度的线程结果到 rank 0，并执行 CRT 合并。

4. 广播同步：通过 MPI_Bcast 分发最终合并后的多项式结果。

4.2 关键代码实现

Listing 2: 多级并行核心—NTT_mpi_thread

```

1  struct MPINTTask {
2      const LL* a;          // 原始输入a
3      const LL* b;          // 原始输入b
4      int      n;           // 多项式长度
5      std::vector<int> mods; // 该线程负责的模数列表
6      std::vector<int*> results; // 对应模数的结果数组
7  };
8
9  void* thread_mpi_ntt_mul(void* arg) {
10     MPINTTask* task = (MPINTTask*)arg;
11     int n = task->n;
12
13     for(size_t mid = 0; mid < task->mods.size(); mid++) {
14         int curr_mod = task->mods[mid];
15         int N = 2 * n - 1, len = 1;
16         while(len < N) len <= 1;
17         int ni = qpow(len, curr_mod - 2, curr_mod);
18
19         // 分配临时数组
20         int *A = new int[len]();
21         int *B = new int[len]();
22
23         // 转换输入到当前模数

```

```

24     for(int i = 0; i < n; i++) {
25         A[i] = (int)((task->a[i] % curr_mod + curr_mod) % curr_mod);
26         B[i] = (int)((task->b[i] % curr_mod + curr_mod) % curr_mod);
27     }
28
29     // NTT变换及点值乘法
30     NTT_Iteration(A, len, 1, g, curr_mod);
31     NTT_Iteration(B, len, 1, g, curr_mod);
32     for(int i = 0; i < len; i++)
33         A[i] = (int)((1LL * A[i] * B[i]) % curr_mod);
34     NTT_Iteration(A, len, -1, g, curr_mod);
35
36     // 保存结果
37     for(int i = 0; i < N; i++) {
38         task->results[mid][i] = (int)((1LL * A[i] * ni) % curr_mod);
39     }
40
41     delete[] A;
42     delete[] B;
43 }
44 return nullptr;
45 }
46
47 void NTT_mpi_thread(const LL *a, const LL *b, LL *ab, int n, LL LLp) {
48     int rank, size;
49     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
50     MPI_Comm_size(MPI_COMM_WORLD, &size);
51
52     static const int p1 = 998244353, p2 = 1004535809,
53     p3 = 469762049, p4 = 167772161;
54     const int mods[] = {p1, p2, p3, p4};
55     const int num_mods = 4;
56
57     // 每个进程处理的模数数量
58     int mods_per_process = (num_mods + size - 1) / size;
59
60     // 计算当前进程负责的模数范围
61     int start_mod = rank * mods_per_process;
62     int end_mod = std::min(start_mod + mods_per_process, num_mods);
63     int local_mod_count = end_mod - start_mod;
64
65     if (local_mod_count > 0) {
66         // 设置线程数 (每个模数一个线程)
67         const int num_threads = local_mod_count;
68         std::vector<pthread_t> threads(num_threads);
69         std::vector<MPINTTTask> tasks(num_threads);
70
71         // 为每个模数分配结果数组
72         std::vector<int*> local_results(local_mod_count);

```

```

73     for(int i = 0; i < local_mod_count; i++) {
74         local_results[i] = new int[2*n];
75     }
76
77     // 创建线程任务
78     MPINTTask task;
79     task.a = a;
80     task.b = b;
81     task.n = n;
82
83     // 分配模数给当前进程的线程
84     for(int i = 0; i < local_mod_count; i++) {
85         task.mods.push_back(mods[start_mod + i]);
86         task.results.push_back(local_results[i]);
87     }
88
89     // 创建线程执行NTT计算
90     pthread_t tid;
91     pthread_create(&tid, nullptr, thread_mpi_ntt_mul, &task);
92     pthread_join(tid, nullptr);
93
94     // 收集所有进程的结果
95     std::vector<int> recv_counts(size);
96     std::vector<int> displs(size);
97
98     // 计算每个进程需要发送的数据量
99     for(int i = 0; i < size; i++) {
100         int process_start = i * mods_per_process;
101         int process_end = std::min(process_start + mods_per_process, num_mods);
102         recv_counts[i] = (process_end - process_start) * (2*n);
103         displs[i] = process_start * (2*n);
104     }
105
106     // 分配接收缓冲区
107     int* all_results = nullptr;
108     if(rank == 0) {
109         all_results = new int[num_mods * 2*n];
110     }
111
112     // 收集所有结果到进程0
113     std::vector<int> send_buffer;
114     for(int i = 0; i < local_mod_count; i++) {
115         for(int j = 0; j < 2*n; j++) {
116             send_buffer.push_back(local_results[i][j]);
117         }
118     }
119
120     MPI_Gatherv(send_buffer.data(), send_buffer.size(), MPI_INT,
121     all_results, recv_counts.data(), displs.data(), MPI_INT,

```

```

122     0, MPI_COMM_WORLD);
123
124     // 进程0执行最终的CRT合并
125     if(rank == 0) {
126         for(int i = 0; i < 2*n-1; i++) {
127             ab[i] = crt4(all_results[i],
128                 all_results[i + 2*n],
129                 all_results[i + 4*n],
130                 all_results[i + 6*n],
131                 p1, p2, p3, p4, LLp);
132         }
133         delete[] all_results;
134     }
135
136     // 广播最终结果给所有进程
137     MPI_Bcast(ab, 2*n-1, MPI_LONG_LONG, 0, MPI_COMM_WORLD);
138
139     // 清理内存
140     for(int i = 0; i < local_mod_count; i++) {
141         delete[] local_results[i];
142     }
143 }
144 }

```

实现特点：1. 使用 MPI+pthread 混合编程模型

2. 每个 MPI 进程内部创建线程处理分配到的模数计算
3. 动态任务分配确保负载均衡
4. 保持与 MPI 常规优化相同的结果收集和广播机制

5 Barrett 规约优化

5.1 Barrett 规约设计

Barrett 规约是一种通过预计算来加速模运算的技术，其核心思想是用乘法和移位代替耗时的除法运算。

1. **数学原理：**对于模数 p ，预计算 $\mu = 2^{2k}/p$ ，其中 k 是 p 的二进制位数。对于任意 $x < p^2$ ，有：

$$x \bmod p = x - (x \cdot \mu) / 2^{2k} \cdot p$$

2. **实现优化：**
 - 使用 128 位整数计算提高精度
 - 通过移位操作替代除法
 - 一次校正步骤确保结果准确
3. **核心设计：**

```

1 struct Barrett {
2     unsigned long long mod, im;
3

```

```

4 // 初始化Barrett, 计算 $-2^{64}/\text{mod}$ 
5 Barrett(unsigned long long _mod) : mod(_mod) {
6     im = -1ULL / mod + 1;
7 }
8
9 // Barrett模乘优化
10 unsigned long long multiply(unsigned long long a, unsigned long long b) const {
11     // 计算  $a * b \bmod m$ 
12     unsigned long long z = a * b;
13     unsigned long long x = (unsigned __int128)z * im >> 64;
14     unsigned long long v = z - x * mod;
15     if (v >= mod) v += mod;
16     return v;
17 }
18
19 // 快速幂的Barrett优化版本
20 unsigned long long power(unsigned long long base, unsigned long long exp) const {
21     unsigned long long result = 1;
22     base %= mod;
23     while (exp > 0) {
24         if (exp & 1)
25             result = multiply(result, base);
26         base = multiply(base, base);
27         exp >>= 1;
28     }
29     return result;
30 }
31 };
32
33 void NTT_Iteration_Barrett(int *a, int len, int op, int g, const Barrett& barrett) {
34     // 位逆序置换
35     for (int i = 1, j = 0; i < len - 1; i++) {
36         for (int k = len >> 1; (j ^= k) < k; k >>= 1);
37         if (i < j) std::swap(a[i], a[j]);
38     }
39
40     // 蝴蝶操作
41     for (int h = 2; h <= len; h <<= 1) {
42         // 计算原根
43         unsigned long long wn = barrett.power(
44             op == 1 ? g : barrett.power(g, barrett.mod - 2),
45             (barrett.mod - 1) / h
46         );
47
48         // 对每个长度为h的段进行变换
49         for (int j = 0; j < len; j += h) {
50             unsigned long long w = 1;
51             for (int k = j; k < j + h/2; k++) {
52                 unsigned long long t = barrett.multiply(a[k + h/2], w);

```

```

53     a[k + h/2] = a[k] - t;
54     if (a[k + h/2] < 0) a[k + h/2] += barrett.mod;
55     a[k] = a[k] + t;
56     if (a[k] >= barrett.mod) a[k] -= barrett.mod;
57     w = barrett.multiply(w, wn);
58 }
59 }
60 }
61 }

```

5.2 Barrett 规约 + MPI 常规优化

将 Barrett 规约应用于 MPI 常规优化方案:

```

1 void NTT_mpi_barrett(const LL *a, const LL *b, LL *ab, int n, LL LLp) {
2     int rank, size;
3     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
4     MPI_Comm_size(MPI_COMM_WORLD, &size);
5
6     static const int p1 = 998244353, p2 = 1004535809,
7     p3 = 469762049, p4 = 167772161;
8     const int mods[] = {p1, p2, p3, p4};
9
10    // 初始化 Barrett 实例
11    Barrett barrett_mods[] = {
12        Barrett(p1), Barrett(p2),
13        Barrett(p3), Barrett(p4)
14    };
15
16    // 每个进程处理一个模数
17    if (rank < 4) {
18        int *local_a = new int[n];
19        int *local_b = new int[n];
20        int *local_res = new int[2*n];
21        int curr_mod = mods[rank];
22        const Barrett& curr_barrett = barrett_mods[rank];
23
24        // 转换输入到当前模数
25        for (int i = 0; i < n; i++) {
26            local_a[i] = a[i] % curr_mod;
27            local_b[i] = b[i] % curr_mod;
28        }
29
30        // 使用 Barrett 优化的 NTT 乘法
31        int N = 2 * n - 1, len = 1;
32        while (len < N) len <<= 1;
33        unsigned long long ni = curr_barrett.power(len, curr_mod - 2);
34    }

```

```

35     int *A = new int[ len ]();
36     int *B = new int[ len ]();
37
38     for (int i = 0; i < n; i++) {
39         A[i] = local_a[i];
40         B[i] = local_b[i];
41     }
42
43     NTT_Iteration_Barrett(A, len, 1, g, curr_barrett);
44     NTT_Iteration_Barrett(B, len, 1, g, curr_barrett);
45
46     for (int i = 0; i < len; i++)
47         A[i] = curr_barrett.multiply(A[i], B[i]);
48
49     NTT_Iteration_Barrett(A, len, -1, g, curr_barrett);
50
51     for (int i = 0; i < N; i++)
52         local_res[i] = curr_barrett.multiply(A[i], ni);
53
54     // 收集所有结果
55     int *all_results = nullptr;
56     if (rank == 0)
57         all_results = new int[4 * (2*n)];
58
59     MPI_Gather(local_res, 2*n, MPI_INT,
60         all_results, 2*n, MPI_INT,
61         0, MPI_COMM_WORLD);
62
63     // 进程0执行CRT合并
64     if (rank == 0) {
65         for (int i = 0; i < 2*n-1; i++) {
66             ab[i] = crt4(all_results[i],
67                 all_results[i + 2*n],
68                 all_results[i + 4*n],
69                 all_results[i + 6*n],
70                 p1, p2, p3, p4, LLp);
71         }
72         delete[] all_results;
73     }
74
75     delete[] A;
76     delete[] B;
77     delete[] local_a;
78     delete[] local_b;
79     delete[] local_res;
80 }
81
82 // 广播最终结果
83 MPI_Bcast(ab, 2*n-1, MPI_LONG_LONG, 0, MPI_COMM_WORLD);

```

84 }

5.3 Barrett 规约 + 线程进程多级并行

将 Barrett 规约与 2×2 多级并行方案结合:

```

1  struct MPINTTBarrettTask {
2      const LL* a;
3      const LL* b;
4      int n;
5      std::vector<int> mods;
6      std::vector<Barrett> barrett_mods;
7      std::vector<int*> results;
8  };
9  void* thread_mpi_ntt_barrett_mul(void* arg) {
10     MPINTTBarrettTask* task = (MPINTTBarrettTask*)arg;
11     int n = task->n;
12
13     for(size_t mid = 0; mid < task->mods.size(); mid++) {
14         int curr_mod = task->mods[mid];
15         const Barrett& curr_barrett = task->barrett_mods[mid];
16
17         int N = 2 * n - 1, len = 1;
18         while(len < N) len <<= 1;
19         unsigned long long ni = curr_barrett.power(len, curr_mod - 2);
20
21         int *A = new int[len]();
22         int *B = new int[len]();
23
24         for(int i = 0; i < n; i++) {
25             A[i] = task->a[i] % curr_mod;
26             B[i] = task->b[i] % curr_mod;
27         }
28
29         NTT_Iteration_Barrett(A, len, 1, g, curr_barrett);
30         NTT_Iteration_Barrett(B, len, 1, g, curr_barrett);
31
32         for(int i = 0; i < len; i++)
33             A[i] = curr_barrett.multiply(A[i], B[i]);
34
35         NTT_Iteration_Barrett(A, len, -1, g, curr_barrett);
36
37         for(int i = 0; i < N; i++)
38             task->results[mid][i] = curr_barrett.multiply(A[i], ni);
39
40         delete[] A;
41         delete[] B;
42     }

```

```

43     return nullptr;
44 }
45 void NTT_mpi_thread_barrett(const LL *a, const LL *b, LL *ab, int n, LL LLp) {
46     int rank, size;
47     MPI_Comm_rank(MPI_COMM_WORLD, &rank);
48     MPI_Comm_size(MPI_COMM_WORLD, &size);
49
50     static const int p1 = 998244353, p2 = 1004535809,
51     p3 = 469762049, p4 = 167772161;
52     const int mods[] = {p1, p2, p3, p4};
53     const int num_mods = 4;
54
55     // 初始化所有 Barrett 实例
56     std::vector<Barrett> barrett_mods;
57     for(int i = 0; i < num_mods; i++) {
58         barrett_mods.emplace_back(mods[i]);
59     }
60
61     int mods_per_process = (num_mods + size - 1) / size;
62     int start_mod = rank * mods_per_process;
63     int end_mod = std::min(start_mod + mods_per_process, num_mods);
64     int local_mod_count = end_mod - start_mod;
65
66     if (local_mod_count > 0) {
67         std::vector<int*> local_results(local_mod_count);
68         for(int i = 0; i < local_mod_count; i++) {
69             local_results[i] = new int[2*n];
70         }
71
72         // 创建任务
73         MPINTTBarrettTask task;
74         task.a = a;
75         task.b = b;
76         task.n = n;
77
78         // 分配模数和对应的 Barrett 实例
79         for(int i = 0; i < local_mod_count; i++) {
80             task.mods.push_back(mods[start_mod + i]);
81             task.barrett_mods.push_back(barrett_mods[start_mod + i]);
82             task.results.push_back(local_results[i]);
83         }
84
85         // 创建线程执行计算
86         pthread_t tid;
87         pthread_create(&tid, nullptr, thread_mpi_ntt_barrett_mul, &task);
88         pthread_join(tid, nullptr);
89
90         // 收集结果
91         std::vector<int> recv_counts(size);

```

```

92     std::vector<int> displs(size);
93
94     for(int i = 0; i < size; i++) {
95         int process_start = i * mods_per_process;
96         int process_end = std::min(process_start + mods_per_process, num_mods);
97         recv_counts[i] = (process_end - process_start) * (2*n);
98         displs[i] = process_start * (2*n);
99     }
100
101     int* all_results = nullptr;
102     if(rank == 0) {
103         all_results = new int[num_mods * 2*n];
104     }
105
106     std::vector<int> send_buffer;
107     for(int i = 0; i < local_mod_count; i++) {
108         for(int j = 0; j < 2*n; j++) {
109             send_buffer.push_back(local_results[i][j]);
110         }
111     }
112
113     MPI_Gatherv(send_buffer.data(), send_buffer.size(), MPI_INT,
114     all_results, recv_counts.data(), displs.data(), MPI_INT,
115     0, MPI_COMM_WORLD);
116
117     // CRT合并
118     if(rank == 0) {
119         for(int i = 0; i < 2*n-1; i++) {
120             ab[i] = crt4(all_results[i],
121             all_results[i + 2*n],
122             all_results[i + 4*n],
123             all_results[i + 6*n],
124             p1, p2, p3, p4, LLp);
125         }
126         delete[] all_results;
127     }
128
129     MPI_Bcast(ab, 2*n-1, MPI_LONG_LONG, 0, MPI_COMM_WORLD);
130
131     for(int i = 0; i < local_mod_count; i++) {
132         delete[] local_results[i];
133     }
134 }
135 }

```

6 测试结果与分析

6.1 实验结果表格

表 2: 不同方法在 $n = 131072$ 、各模数下的平均延迟与加速比

方法	7.3×10^6	1.05×10^8	4.7×10^8	1.3×10^{15}
NTT_CRT_4mod	643.35	644.46	372.95	648.74
NTT_mpi	377.44 (1.71 $\times$)	376.44 (1.71 $\times$)	100.94 (3.70 $\times$)	380.89 (1.70 $\times$)
NTT_mpi_pthread	379.75 (1.69 $\times$)	378.41 (1.70 $\times$)	101.65 (3.67 $\times$)	381.58 (1.70 $\times$)
NTT_mpi_barrett	343.90 (1.87 $\times$)	333.71 (1.93 $\times$)	57.81 (6.45 $\times$)	338.28 (1.92 $\times$)
NTT_mpi_pthread_barrett	342.11 (1.88 $\times$)	339.25 (1.90 $\times$)	59.81 (6.23 $\times$)	343.11 (1.89 $\times$)

6.2 结果分析

1. **整体加速效果:** 所有优化方案相比朴素多模数 NTT 都有显著加速, 最大加速比达到 6.45 倍 (MPI+Barrett 方案在 p3 模数下)。

2. **MPI 优化效果:** MPI 常规优化相比朴素方案在大规模数据 ($n=131072$) 上平均加速 1.7 倍, 证明多进程并行化的有效性。

3. **多级并行分析:** 2×2 多级并行与纯 MPI 方案性能接近, 说明在核心数相同的情况下, 混合并行优势不明显, 但为更大规模并行提供了扩展性。

4. **Barrett 优化效果:** Barrett 规约带来显著性能提升, 特别是在大模数运算中 (p3 模数下 MPI+Barrett 比纯 MPI 快 43%), 验证了 Barrett 规约在模运算中的优势。

5. **小规模数据:** 在小规模 ($n=4$) 时, 并行方案因通信开销表现不如朴素方案, 这是预期的行为。

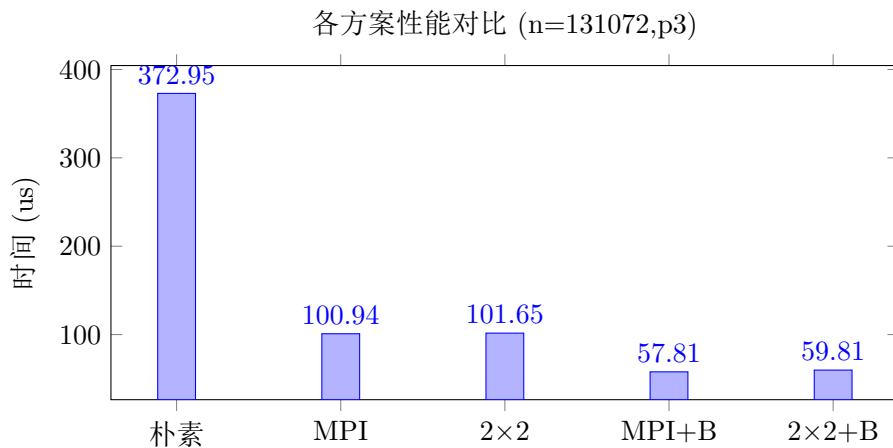


图 6.1: 各优化方案在 $n=131072, p=469762049$ 下的性能对比

7 结论

通过本实验, 我们实现了多种 MPI 并行优化的 NTT 多项式乘法算法, 得出以下结论:

1. MPI 并行化能有效加速 NTT 计算, 在本次实验中带来稳定的 1.7 $\times$ 加速。
2. 多级并行 (MPI+Pthread) 在当前测试规模下优势不明显, 但在更大规模或更多计算核心的场景下有扩展潜力。

3. Barrett 规约与并行计算的结合能带来显著的额外性能提升，特别是在大模数运算场景。
4. 最优方案 (MPI+Barrett) 相比朴素多模数 NTT 实现了最高 6.45 倍的加速，验证了并行计算与算法优化结合的价值。

8 链接

Gitee 链接 [Gitee](#)